

PHASE 2 · WEEK 5 · REGRESSION & CLASSIFICATION

Linear & Logistic Regression

Theory · Mathematical Derivations · Loss Functions · Optimization

A Complete, Beginner-Friendly Study Guide

From first principles to production-ready models — every idea explained in plain language first, then in mathematical notation, then connected to working code.

Companion to the “Linear & Logistic Regression” slide deck and Jupyter notebook

This guide assumes no prior background in machine learning or advanced mathematics beyond basic algebra. Every derivation is written out in full, step by step.

Contents

How to Use This Guide	3
1 Foundations: What You Need Before We Start	4
1.1 What Does “Learning From Data” Actually Mean?	4
1.2 Notation You Will See Throughout	5
1.2.1 Vectors and the Dot Product	5
1.2.2 Summation Notation	5
1.2.3 Derivatives and Gradients, Intuitively	6
1.2.4 Probability Basics: Density and Likelihood	6
2 Linear Regression	7
2.1 Model Representation: The Linear Regression Hypothesis	7
2.2 Assumptions Behind the Model	8
2.3 Why Squared Error? A Probabilistic View	8
2.3.1 Derivation: From Maximum Likelihood to Mean Squared Error	8
2.4 The Cost Function	9
2.5 Vectorized Cost & Gradient	10
2.6 The Normal Equation: A Closed-Form Solution	11
2.7 Gradient Descent: An Iterative Solution	11
2.7.1 Deriving the Update Rule	12
2.8 Convergence Theory: Why Gradient Descent Actually Works	12
2.8.1 Part I — Why a Stationary Point Is <i>the</i> Global Minimum	13
2.8.2 Part II — The Descent Lemma: Every Step Decreases the Cost	13
2.8.3 Part III — Putting It Together: Convergence to θ^*	14
2.9 Gradient Descent Variants & Choosing a Learning Rate	15
2.10 Regularization: Ridge, Lasso, and Elastic Net	15
2.10.1 Ridge Regression (L2 Penalty)	15
2.10.2 Lasso (L1 Penalty) and Elastic Net	16
2.11 Evaluating a Regression Model	16
3 Logistic Regression	18
3.1 Why Not Just Use Linear Regression for Classification?	18
3.2 The Sigmoid (Logistic) Function	18
3.3 Model Representation & Log-Odds	19
3.4 Maximum Likelihood: The Bernoulli Model	20
3.5 From Log-Likelihood to Binary Cross-Entropy Loss	20
3.6 Why Cross-Entropy Makes Intuitive Sense	21
3.7 The Gradient — A Beautifully Simple Result	21
3.8 Optimization & the Convexity Guarantee	22
3.9 Regularized & Multiclass (Softmax) Logistic Regression	22
3.9.1 Extending to More Than Two Classes: Softmax Regression	23
3.10 Evaluating a Classification Model	23

4	Synthesis: Linear and Logistic Regression, Side by Side	25
4.1	Comparison Table	25
4.2	Key Takeaways	25
4.3	From Theory to a Working Project	26
	Appendix A: Notation & Symbol Glossary	27
	Appendix B: Frequently Asked Questions	28
	Appendix C: Further Study	29

How to Use This Guide

This guide is the written companion to the “Linear & Logistic Regression” slide deck and its accompanying Jupyter notebook. It is written for students who have some basic algebra but **little or no prior background in machine learning, calculus, or probability**. Every idea follows the same three-step pattern:

1. **Plain-English intuition** — what is this idea really saying, in everyday words?
2. **Mathematical notation** — the precise formula, derived step by step, with no steps skipped.
3. **Connection to code** — a pointer to where this exact idea appears in the companion notebook.

Throughout the guide you will see colored boxes that flag different kinds of content:

Key Insight

The single most important idea in a section — the takeaway you should remember even if you forget everything else.

Plain-English Intuition

A plain-language restatement of a formula, often with an everyday analogy.

Worked Example

A fully worked numerical example, with real numbers plugged into the formula.

Derivation

A step-by-step mathematical derivation, where every algebraic step is shown (nothing is left as “it can be shown that...”).

Watch Out

A common mistake or misconception to watch out for.

Chapter 1

Foundations: What You Need Before We Start

1.1 What Does “Learning From Data” Actually Mean?

Suppose you want to predict the price of a house. You don’t know the exact formula the market uses, but you have historical data: for many houses, you know their size, number of bedrooms, age — and the price they sold for. Machine learning, for this kind of problem, is just a four-step recipe:

1. **Choose a hypothesis** — assume a simple mathematical *shape* for the relationship between inputs and output (e.g. “price is a weighted sum of features”).
2. **Define a cost function** — a single number that measures “how wrong” the model currently is, given some choice of parameters.
3. **Optimize** — search for the parameters that make the cost as small as possible. This search process is called *training*.
4. **Evaluate** — check how well the trained model performs on data it has never seen before.

This guide covers two of the most fundamental models that follow exactly this recipe: **Linear Regression** (which predicts a continuous number, like a price) and **Logistic Regression** (which predicts a category or probability, like “spam or not spam”). Despite solving different kinds of problems, the two models share almost the exact same mathematical skeleton — spotting that shared skeleton is one of the main goals of this guide.

1.2 Notation You Will See Throughout

Symbol	Meaning
$x_1, x_2, \dots, x_n$	The n input features (independent variables) of one example
y	The true target value (what we're trying to predict)
$\hat{y}$	The model's predicted value for y
$\theta_0, \theta_1, \dots, \theta_n$	The model's parameters (weights), including bias θ_0
m	The number of training examples in our dataset
n	The number of input features
$x^{(i)}$	The feature vector of the i -th training example (superscript = example index)
x_j	The j -th feature (subscript = feature index)
X	The design matrix — all m examples stacked into one $m \times (n+1)$ matrix
$\theta^T x$	The dot product of parameter vector θ and feature vector x
$J(\theta)$	The cost function — “how wrong” the model is, as a function of θ
α	The learning rate — how big a step we take during optimization
$\nabla_{\theta} J(\theta)$	The gradient of J — a vector of partial derivatives, one per parameter
$\sigma(z)$	The sigmoid function, used in logistic regression
λ	The regularization strength

1.2.1 Vectors and the Dot Product

A **vector** is just an ordered list of numbers. If $\theta = (\theta_0, \theta_1, \theta_2)$ and $x = (x_0, x_1, x_2)$, their **dot product** $\theta^T x$ is defined as:

$$\theta^T x = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2$$

Plain-English Intuition

The dot product is just a weighted sum: multiply each pair of matching numbers together, then add up all the results. This is exactly what a spreadsheet's SUMPRODUCT function computes. In machine learning, we use it to combine several input features into a single number using learned weights.

1.2.2 Summation Notation

The symbol $\sum_{i=1}^m a_i$ simply means “add up a_i for every value of i from 1 to m ”:

$$\sum_{i=1}^m a_i = a_1 + a_2 + \dots + a_m$$

You will see this constantly — it is how we express “average this quantity over every training example.”

1.2.3 Derivatives and Gradients, Intuitively

A **derivative** measures how fast a function's output changes as you nudge its input slightly. If you imagine a cost function $J(\theta)$ as the height of a hilly landscape as you walk along the θ -axis, the derivative $\frac{dJ}{d\theta}$ at any point tells you the *slope* of the ground right there: positive means uphill, negative means downhill, and zero means flat (a potential valley bottom or hilltop).

When J depends on *several* parameters $\theta_0, \theta_1, \dots, \theta_n$ at once, we compute a separate **partial derivative** $\frac{\partial J}{\partial \theta_j}$ for each parameter — “how does J change if I nudge just θ_j , holding everything else fixed?” Stacking all of these partial derivatives into one vector gives the **gradient**, written $\nabla_{\theta} J(\theta)$. The gradient always points in the direction of *steepest increase* of J — which is exactly why, to *decrease* the cost, optimization algorithms move in the *opposite* direction of the gradient. This single idea is the engine behind nearly all of machine learning, and we will use it constantly in Chapters 2 and 3.

1.2.4 Probability Basics: Density and Likelihood

A **probability density function (PDF)**, written $p(y)$, describes how likely different values of a continuous random variable y are. The most famous PDF is the **Gaussian (Normal) distribution**:

$$p(y) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - \mu)^2}{2\sigma^2}\right)$$

where μ is the mean (center of the bell curve) and σ^2 is the variance (how spread out it is). You don't need to memorize this formula — just recognize its shape: a bell curve, tallest at μ , spreading out symmetrically, controlled by σ^2 .

Likelihood is the same mathematical object as a density or probability, but with a change in perspective: instead of asking “given fixed parameters, how likely is this data?”, likelihood asks “given this fixed, observed data, how does the probability change as I vary the parameters?”

Maximum Likelihood Estimation (MLE) is the principle of choosing whichever parameters make the observed data as probable as possible. This single idea — MLE — is what generates *both* of the loss functions we will derive in this guide (Mean Squared Error in Chapter 2, and Cross-Entropy in Chapter 3), which is why they aren't arbitrary formulas someone made up, but the mathematically correct consequence of an assumption about noise.

Chapter 2

Linear Regression

Linear regression predicts a **continuous** outcome — a number like a price, a temperature, or a salary. This chapter builds the model from scratch: first the equation itself, then the deep question of *why* we measure error the way we do, then how to actually find the best parameters, and finally how to guard against overfitting and how to judge whether the model is any good.

2.1 Model Representation: The Linear Regression Hypothesis

Linear regression models the target y as a weighted sum of the input features, plus one constant “baseline” term:

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

- $\theta_0, \theta_1, \dots, \theta_n$ — the **parameters** (weights) learned from data.
- $x_1, \dots, x_n$ — the **input features** (independent variables), e.g. square footage, number of bedrooms.
- θ_0 is called the **bias** or **intercept** — the predicted value when every feature is zero.

If we define an extra “feature” $x_0 = 1$ for every example, the whole hypothesis collapses into a single dot product:

$$h_{\theta}(x) = \theta^T x$$

Plain-English Intuition

This is exactly what a spreadsheet’s SUMPRODUCT function computes: multiply each feature by its weight, and add everything up. The bias term θ_0 is just a weight attached to a feature that’s always equal to 1 — a constant amount added no matter what the other features are.

Goal: find the parameter vector θ that makes $h_{\theta}(x^{(i)})$ as close as possible to the true $y^{(i)}$, for every training example $i = 1, \dots, m$.

Worked Example

Suppose a (deliberately simple) trained model for house prices is $h_{\theta}(x) = 15,000 + 120 \cdot (\text{sqft}) + 8,000 \cdot (\text{bedrooms})$. For a 1,800 sqft house with 3 bedrooms:

$$h_{\theta}(x) = 15,000 + 120(1800) + 8,000(3) = 15,000 + 216,000 + 24,000 = \$255,000$$

Every one-square-foot increase adds \$120 to the predicted price; every extra bedroom adds \$8,000 — holding everything else fixed.

2.2 Assumptions Behind the Model

Linear regression works best (and its statistical guarantees are trustworthy) when the following conditions roughly hold. It's worth knowing these even before we discuss optimization, because later we will actually check several of them on real trained-model residuals.

Assumption	Plain-English meaning
Linearity	The relationship between features and target is (approximately) linear in the parameters θ .
Independence	Observations (and their errors) don't influence one another.
Homoscedasticity	Residuals have roughly constant variance across all levels of the fitted value — no “funnel” shape.
Normality of errors	Residuals ε are approximately normally distributed — this is exactly what justifies using Mean Squared Error, as the next section will show.
No multicollinearity	Features aren't highly correlated with one another, so the estimated θ values stay numerically stable.
Few/no extreme outliers	A handful of extreme points can disproportionately pull the least-squares fit toward them.

2.3 Why Squared Error? A Probabilistic View

Why do we minimize *squared* error instead of, say, absolute error or some other measure? It isn't an arbitrary convention — it falls directly out of a simple, explicit assumption about noise.

Assume every observed target is the “true” linear signal plus random Gaussian (bell-curve) noise:

$$y^{(i)} = \theta^T x^{(i)} + \varepsilon^{(i)}, \quad \varepsilon^{(i)} \sim \mathcal{N}(0, \sigma^2)$$

Equivalently, $y^{(i)}$ given $x^{(i)}$ is normally distributed, centered on the line $\theta^T x^{(i)}$:

$$p(y^{(i)} | x^{(i)}; \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$$

Plain-English Intuition

This says: “most houses sell close to the price our line predicts, with small random deviations in either direction being common and large deviations being rare” — exactly the bell-curve shape you'd expect from countless small, unmodeled factors (a slightly motivated buyer, a slightly stale listing, and so on) nudging the price up or down.

2.3.1 Derivation: From Maximum Likelihood to Mean Squared Error

Maximum Likelihood Estimation (MLE) says: choose the θ that makes the data we actually observed as probable as possible under the model.

Step 1 — Likelihood of the full dataset

Assuming the m examples are independent and identically distributed (i.i.d.), the joint likelihood is the product of each example's individual density:

$$L(\theta) = \prod_{i=1}^m p(y^{(i)} | x^{(i)}; \theta) = \prod_{i=1}^m \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$$

Step 2 — Take the log-likelihood

Products of many small numbers are numerically painful and hard to differentiate, so we take the natural log. Because log is strictly increasing, whichever θ maximizes $L(\theta)$ also maximizes $\log L(\theta)$ — so this doesn't change the answer, it just makes the algebra tractable. Using $\log(ab) = \log a + \log b$, the product becomes a sum:

$$\begin{aligned} \ell(\theta) = \log L(\theta) &= \sum_{i=1}^m \log \left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right) \right] \\ &= m \log \frac{1}{\sqrt{2\pi\sigma^2}} - \frac{1}{2\sigma^2} \sum_{i=1}^m (y^{(i)} - \theta^T x^{(i)})^2 \end{aligned}$$

Notice the first term, $m \log \frac{1}{\sqrt{2\pi\sigma^2}}$, does not depend on θ at all — it's a constant as far as our search over θ is concerned.

Step 3 — Maximizing $\ell(\theta)$ is equivalent to minimizing squared error

Since the first term is constant in θ , and $\sigma^2 > 0$ is just a fixed positive scaling factor, maximizing $\ell(\theta)$ with respect to θ is mathematically identical to **minimizing**:

$$\sum_{i=1}^m (y^{(i)} - \theta^T x^{(i)})^2$$

(Flipping the sign turns “maximize –stuff” into “minimize stuff”, and dividing by the positive constant $2\sigma^2$ doesn't change which θ is optimal.)

Key Insight

“Least squares” isn't an arbitrary choice — it falls directly out of assuming Gaussian noise. If you believed your noise followed a *different* distribution, MLE would hand you a *different* loss function. This is exactly what happens in Chapter 3: assuming Bernoulli-distributed labels instead of Gaussian noise produces the cross-entropy loss instead of squared error — the same principle, a different noise assumption.

2.4 The Cost Function

Averaging the squared errors over all m training examples (and, by convention, including an extra factor of $\frac{1}{2}$) gives the **Mean Squared Error (MSE) cost function**:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

Let's unpack every piece of this formula:

- **The $\frac{1}{2}$** is pure notational convenience — it cancels neatly when we differentiate (see the Gradient Descent derivation below). It does not change *which* θ minimizes J .
- **The $\frac{1}{m}$** averages the error over all training examples, so the cost is comparable across datasets of different sizes.
- **Squaring** penalizes larger errors disproportionately (an error of 10 hurts $100\times$ more than an error of 1) and keeps the function smooth and differentiable everywhere, which matters for the optimization methods discussed later in this chapter.
- $J(\theta)$ is a **convex quadratic bowl** in θ — it has exactly *one* global minimum, with no other local minima to get stuck in. The Convergence Theory section later in this chapter makes this claim rigorous.

2.5 Vectorized Cost & Gradient

Working with individual summations is fine for understanding, but real implementations use matrix notation for both clarity and computational speed. Stack all m examples into a **design matrix** $X \in \mathbb{R}^{m \times (n+1)}$ (one row per example, one column per feature, plus a column of 1s for the bias) and targets $y \in \mathbb{R}^m$. The cost becomes:

$$J(\theta) = \frac{1}{2m} (X\theta - y)^T (X\theta - y)$$

Deriving the Gradient in Matrix Form

We want $\nabla_{\theta} J(\theta)$. Start by expanding the quadratic form:

$$(X\theta - y)^T (X\theta - y) = (X\theta)^T (X\theta) - (X\theta)^T y - y^T (X\theta) + y^T y = \theta^T X^T X \theta - 2\theta^T X^T y + y^T y$$

(using the fact that $(X\theta)^T y = y^T (X\theta)$, since both are scalars, so we can combine the two middle terms). Now differentiate term by term using two standard matrix-calculus identities:

- $\frac{\partial(\theta^T A \theta)}{\partial \theta} = 2A\theta$ for any symmetric matrix A (here $A = X^T X$, which is always symmetric).
- $\frac{\partial(b^T \theta)}{\partial \theta} = b$ for any constant vector b (here $b = X^T y$).

Applying these:

$$\nabla_{\theta} \left[\theta^T X^T X \theta - 2\theta^T X^T y + y^T y \right] = 2X^T X \theta - 2X^T y$$

Dividing by $2m$ to match the $J(\theta) = \frac{1}{2m}(\cdot)$ normalization:

$$\nabla_{\theta} J(\theta) = \frac{1}{m} \left[X^T X \theta - X^T y \right] = \frac{1}{m} X^T (X\theta - y)$$

Key Insight

This single vectorized gradient expression, $\nabla_{\theta} J(\theta) = \frac{1}{m} X^T (X\theta - y)$, is the engine that drives *both* the closed-form Normal Equation and iterative Gradient Descent, covered next.

2.6 The Normal Equation: A Closed-Form Solution

Setting the gradient from the previous section to zero and solving directly for θ :

Solving $\nabla_{\theta} J(\theta) = 0$

$$\frac{1}{m} X^T (X\theta - y) = 0 \implies X^T X\theta - X^T y = 0 \implies X^T X\theta = X^T y$$

Left-multiplying both sides by $(X^T X)^{-1}$ (assuming this inverse exists):

$$\theta = (X^T X)^{-1} X^T y$$

This gives the *exact* optimum in a single step — no iteration, no learning rate to tune. Table 2.1 compares it against the iterative approach of the next section.

Table 2.1: Normal Equation vs. Gradient Descent

	Normal Equation	Gradient Descent
Learning rate α needed?	No	Yes, must be tuned
Iterative?	No — exact in one step	Yes — needs many iterations
Computational complexity	$O(n^3)$ to invert $X^T X$	$O(n)$ per iteration
Best when	n is small-moderate (roughly $n < 10,000$)	n is large
Feature scaling required?	No	Strongly recommended
Watch out for	$X^T X$ must be invertible — fails with redundant/perfectly correlated features (Ridge, Section 2.10, fixes this)	Approximate; converges toward, but never exactly reaches, the optimum

2.7 Gradient Descent: An Iterative Solution

2.7.1 Deriving the Update Rule

Partial derivative with respect to a single θ_j

Starting from $J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_\theta(x^{(i)}) - y^{(i)})^2$, apply the chain rule. Let $e^{(i)} = h_\theta(x^{(i)}) - y^{(i)}$. Then $J = \frac{1}{2m} \sum_i (e^{(i)})^2$, and:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{2m} \sum_i 2e^{(i)} \cdot \frac{\partial e^{(i)}}{\partial \theta_j}$$

Since $h_\theta(x^{(i)}) = \theta_0 x_0^{(i)} + \theta_1 x_1^{(i)} + \dots$, we have $\frac{\partial e^{(i)}}{\partial \theta_j} = x_j^{(i)}$ (every other term in the sum doesn't involve θ_j , so its derivative is 0). The factor of 2 cancels with the $\frac{1}{2}$:

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Gradient descent repeatedly nudges every θ_j a small step *opposite* to the gradient (the direction of steepest ascent, so the negative gradient is the direction of steepest *descent*):

$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j} \quad (\text{simultaneously, for every } j)$$

- α , the **learning rate**, controls the step size. Too small $\rightarrow$ painfully slow convergence. Too large $\rightarrow$ the algorithm overshoots and can even diverge (Section 2.9 shows this happening).
- “**Simultaneous update**” means every θ_j must be updated using the *old* θ values, not the partially-updated ones — otherwise later partial derivatives would be computed against an inconsistent parameter vector, and the math breaks.

Worked Example

Suppose (with scaled features) we start at $\theta = (0, 0)$, learning rate $\alpha = 0.1$, and a single training example gives gradient $\nabla_\theta J = (4, -2)$. One gradient descent step gives:

$$\theta_0 := 0 - 0.1(4) = -0.4, \quad \theta_1 := 0 - 0.1(-2) = 0.2$$

The parameter with the larger (positive) gradient moved further in the negative direction; the parameter with a negative gradient moved in the positive direction — always opposite the slope, exactly as intended.

2.8 Convergence Theory: Why Gradient Descent Actually Works

This section answers a deeper question: *how do we know gradient descent finds the best solution, and not just some okay solution?* The argument has three parts, each building on the last. This is the most mathematically involved section in the guide — take it slowly, and lean on the intuition boxes.

2.8.1 Part I — Why a Stationary Point Is *the* Global Minimum

A function is **convex** if it always lies on or above every one of its own tangent lines. In one dimension, this means:

$$J(\theta') \geq J(\theta) + \nabla J(\theta) \cdot (\theta' - \theta) \quad \text{for every } \theta, \theta'$$

Plain-English Intuition

Picture standing on the cost bowl at some point θ , and drawing the straight line that's tangent to the bowl there (just touching it, matching its slope). Convexity says: no matter where else on the bowl you look (θ'), the bowl's actual height is never *below* that straight tangent line. A bowl obviously has this property; a wavy, hilly landscape (like a neural network's loss surface) generally does not.

Why this makes optimization easy

Suppose we find a point θ^* where the gradient is exactly zero: $\nabla J(\theta^*) = 0$. Plugging $\theta = \theta^*$ into the convexity inequality above:

$$J(\theta') \geq J(\theta^*) + \underbrace{\nabla J(\theta^*)}_{=0} \cdot (\theta' - \theta^*) = J(\theta^*)$$

This holds for *every* θ' — meaning $J(\theta^*) \leq J(\theta')$ for every possible θ' in the entire parameter space. In other words, θ^* isn't just a local dip; it is **the single best point anywhere**.

Key Insight

For a *non-convex* loss (e.g. a neural network's), a point where the gradient is zero could be a local minimum, a saddle point, or a plateau — nowhere near the best solution. For our convex MSE and (as Chapter 3 will show) cross-entropy costs, there is no such ambiguity: every stationary point is automatically the single best point in the entire parameter space.

2.8.2 Part II — The Descent Lemma: Every Step Decreases the Cost

Convexity alone tells us *where* the best point is, but not that gradient descent will actually *reach* it without overshooting. For this we need one more assumption: that the gradient doesn't change arbitrarily fast — formally, that ∇J is **L -Lipschitz**, meaning $\|\nabla J(\theta_1) - \nabla J(\theta_2)\| \leq L\|\theta_1 - \theta_2\|$ for some constant L .

Plain-English Intuition

This is a “no sudden cliffs” assumption: the slope of the bowl can't change infinitely fast as you move a tiny distance. It rules out pathologically jagged cost surfaces and lets us bound how much the cost can possibly change after one gradient step.

From smoothness to a quadratic upper bound

The L -Lipschitz assumption on the gradient implies a quadratic *upper bound* on J everywhere:

$$J(\theta') \leq J(\theta) + \nabla J(\theta)^T(\theta' - \theta) + \frac{L}{2} \|\theta' - \theta\|^2$$

Now plug in the gradient descent step itself, $\theta' = \theta - \alpha \nabla J(\theta)$, so $\theta' - \theta = -\alpha \nabla J(\theta)$:

$$\begin{aligned} J(\theta - \alpha \nabla J(\theta)) &\leq J(\theta) + \nabla J(\theta)^T(-\alpha \nabla J(\theta)) + \frac{L}{2} \|\alpha \nabla J(\theta)\|^2 \\ &= J(\theta) - \alpha \|\nabla J(\theta)\|^2 + \frac{L\alpha^2}{2} \|\nabla J(\theta)\|^2 = J(\theta) - \alpha \left(1 - \frac{L\alpha}{2}\right) \|\nabla J(\theta)\|^2 \end{aligned}$$

This is the **descent lemma**.

Key Insight

Choosing a safe step size $\alpha \leq 1/L$ (a common, conservative choice) makes $(1 - \frac{L\alpha}{2}) \geq \frac{1}{2}$, a guaranteed positive number. That means the right-hand side above is *strictly less* than $J(\theta)$ whenever $\nabla J(\theta) \neq 0$ — every single gradient descent step is mathematically guaranteed to make progress, never overshooting into a worse region, as long as $\alpha < 2/L$.

2.8.3 Part III — Putting It Together: Convergence to θ^* **Telescoping the per-step decrease**

Summing the descent lemma's guaranteed decrease over T iterations, the terms telescope — each iteration's starting cost cancels the previous iteration's ending cost, leaving only the very first and very last:

$$\sum_{t=0}^{T-1} \alpha \left(1 - \frac{L\alpha}{2}\right) \|\nabla J(\theta_t)\|^2 \leq J(\theta_0) - J(\theta_T) \leq J(\theta_0) - J(\theta^*)$$

The right-hand side is a fixed, finite number (it doesn't grow with T). Since the left-hand side is a sum of T *non-negative* terms bounded above by that finite number, the individual terms $\|\nabla J(\theta_t)\|^2$ must shrink toward zero as $T \rightarrow \infty$ — otherwise the sum would grow without bound.

Key Insight

Combining this with Part I's convexity argument: as the gradient shrinks to zero, θ_t must be approaching a stationary point, and every stationary point of a convex function is the global minimum. So $\theta_t \rightarrow \theta^*$, and gradient descent is guaranteed to converge to the exact best solution — not merely “a good” solution.

For linear regression's MSE specifically, the Lipschitz constant L is exactly the largest eigenvalue of $\frac{1}{m} X^T X$. This connects directly to the companion notebook's preprocessing step: **feature scaling shrinks L** , which is why standardizing features lets you safely use a larger, faster-converging learning rate.

2.9 Gradient Descent Variants & Choosing a Learning Rate

In practice, we rarely compute the gradient over the *entire* dataset at every single step. Three common variants trade off accuracy of the gradient estimate against speed:

Variant	How it works	Trade-off
Batch GD	Uses ALL m examples per update step.	Stable, accurate gradient — but slow per step on large datasets.
Stochastic GD (SGD)	Uses ONE randomly chosen example per update.	Very fast updates, but a noisy, zig-zagging path; needs a decaying α to settle down.
Mini-batch GD	Uses a small batch (e.g. 32–256 examples).	The standard in practice — balances speed and stability.

Choosing a learning rate. There is no formula that hands you the perfect α in advance; instead, practitioners follow a simple experimental protocol:

- Start with a range of candidate values, e.g. 0.3, 0.1, 0.03, 0.01, $\dots$, and plot $J(\theta)$ against iteration number for each.
- For batch gradient descent, $J(\theta)$ should decrease at *every* iteration. If it ever increases, α is too large — pick a smaller value.
- For SGD or mini-batch GD, use a **learning-rate schedule** (gradually decaying α over time) so the noisy updates settle near the true minimum instead of bouncing around it forever.
- Adaptive optimizers (Adam, RMSProp, Adagrad) automate this schedule-tuning process — these are covered in later weeks on neural networks, but they are built on exactly the gradient descent foundation derived above.

2.10 Regularization: Ridge, Lasso, and Elastic Net

When a model has many features (or features that are correlated with each other), it can fit the training data almost perfectly while generalizing poorly to new data — a problem called **overfitting**. Regularization combats this by adding a penalty for large parameter values directly into the cost function.

2.10.1 Ridge Regression (L2 Penalty)

Ridge adds a penalty on the squared magnitude of the weights:

$$J_{\text{ridge}}(\theta) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2 + \lambda \sum_{j=1}^n \theta_j^2$$

This has a modified closed-form solution (derived the same way as the ordinary Normal Equation in Section 2.6, but differentiating the extra penalty term as well):

$$\theta = (X^T X + \lambda I')^{-1} X^T y$$

where I' is the identity matrix with a 0 in the top-left corner, since we typically don't want to regularize (shrink) the bias term θ_0 .

- λ (lambda) controls the regularization strength: $\lambda = 0$ recovers ordinary least squares; a large λ shrinks all θ_j toward zero.
- Shrinking weights reduces **variance** (overfitting) at the cost of a little **bias** — the classic *bias-variance trade-off* that appears throughout machine learning.
- Adding λI always makes $X^T X + \lambda I$ invertible, even when $X^T X$ alone is not — so Ridge also directly fixes the “singular matrix” failure mode of the plain Normal Equation under perfect multicollinearity (Section 2.2).

2.10.2 Lasso (L1 Penalty) and Elastic Net

Lasso penalizes the *absolute value* of the weights instead of their square:

$$J_{\text{lasso}}(\theta) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2 + \lambda \sum_{j=1}^n |\theta_j|$$

	Ridge (L2)	Lasso (L1)
Effect on coefficients	Shrinks smoothly toward zero, rarely exactly zero.	Can shrink coefficients to <i>exactly</i> zero — automatic feature selection.
Best used when	Most features are somewhat useful and should be kept.	You suspect many features are irrelevant.

Plain-English Intuition

Geometrically, Ridge’s penalty region is a smooth circle (in 2D) or sphere (in higher dimensions), while Lasso’s is a diamond with sharp corners. When the elliptical contours of the squared-error cost function expand outward from the unregularized optimum, they are much more likely to first touch the diamond *exactly at one of its corners* — where one coordinate is exactly zero — than to touch a smooth circle at a point where a coordinate is exactly zero. This geometric quirk is the real reason Lasso produces sparse (many-zero) solutions while Ridge does not.

Elastic Net combines both penalties, $\lambda_1 \sum_j |\theta_j| + \lambda_2 \sum_j \theta_j^2$, getting Lasso’s sparsity together with Ridge’s stability — especially useful when features are correlated with each other (in which case Lasso alone tends to arbitrarily pick just one of the correlated features and zero out the rest, which Ridge’s smoother shrinkage avoids).

2.11 Evaluating a Regression Model

Once a model is trained, we need to measure how good it actually is — always on data the model did *not* train on.

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \quad \text{Adjusted } R^2 = 1 - (1 - R^2) \frac{m-1}{m-n-1}$$

$$\text{RMSE} = \sqrt{\frac{1}{m} \sum_i (y_i - \hat{y}_i)^2} \quad \text{MAE} = \frac{1}{m} \sum_i |y_i - \hat{y}_i|$$

- **R^2 (Coefficient of Determination)** measures the fraction of variance in y explained by the model, relative to just always predicting the mean $\bar{y}$. A value close to 1 means the model explains almost all the variation; a value of 0 means it's no better than predicting the average every time; negative values are possible and mean the model is worse than that naive baseline.
- **Adjusted R^2** penalizes adding extra features that don't really help, dividing by degrees of freedom $m - n - 1$ instead of just $m - 1$ — useful for fairly comparing models with different numbers of features.
- **RMSE** is in the same units as y , but squares errors before averaging, so it punishes a few large mistakes more heavily.
- **MAE** is also in the same units as y , treats every error linearly, and is more robust to outliers than RMSE, since errors aren't squared.

Worked Example

Suppose a model predicts house prices with $\text{RMSE} = \$18,500$ and $\text{MAE} = \$14,200$, on houses averaging $\$280,000$. RMSE being noticeably larger than MAE suggests the model has a handful of larger misses dragging up the squared average, even though its *typical* (median-like) error, reflected better by MAE, is smaller. Relative to the $\$280,000$ average price, both errors represent roughly 5–7% of a typical house's value — context that raw dollar figures alone don't convey.

Watch Out

An RMSE or MAE value means very little in isolation — always compare it against the natural scale of y . An RMSE of 5 is excellent if house prices are in the hundreds of thousands, but terrible if you're predicting a percentage between 0 and 1.

Chapter 3

Logistic Regression

Logistic regression predicts a **category** — most simply, a yes/no outcome like “is this email spam?” or “will this customer churn?” Remarkably, as this chapter shows, it reuses almost the entire mathematical skeleton of linear regression from Chapter 2, wrapped inside one new ingredient: a function that squashes any real number into a genuine probability.

3.1 Why Not Just Use Linear Regression for Classification?

Suppose we want to predict a category (e.g. “is this tumor malignant?”, label 0 or 1) instead of a continuous number. Why not simply fit a straight line to the 0/1 labels, as in Chapter 2? Four problems arise:

Problem	Explanation
Unbounded output	$\theta^T x$ ranges over all real numbers, but we need a probability strictly between 0 and 1.
Sensitive to outliers	A single far-away point can shift the decision threshold for every other point.
Poor loss geometry	Squared error against 0/1 labels, composed with a hard threshold, is non-convex and doesn’t penalize confident wrong answers strongly enough.
No probabilistic interpretation	We want $p(y=1 \mid x)$, a genuine probability — not just a raw number.

The fix: pass $\theta^T x$ through a *squashing function* that maps all of $\mathbb{R}$ into $(0, 1)$ — the **sigmoid** function.

3.2 The Sigmoid (Logistic) Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Plain-English Intuition

No matter how large or small the input z is, $\sigma(z)$ always comes out strictly between 0 and 1 — exactly what we need to represent a probability. As z grows large and positive, e^{-z} shrinks toward 0, so $\sigma(z) \rightarrow 1$. As z grows large and negative, e^{-z} explodes, so $\sigma(z) \rightarrow 0$. Right at $z = 0$, $\sigma(0) = \frac{1}{1+1} = 0.5$ exactly — an equally-likely coin flip.

Properties that make the sigmoid perfect for this job:

- $\sigma(z) \rightarrow 1$ as $z \rightarrow +\infty$, and $\sigma(z) \rightarrow 0$ as $z \rightarrow -\infty$ — always bounded in $(0, 1)$.
- $\sigma(0) = 0.5$ exactly — this is the natural decision boundary (predict class 1 if $\sigma(z) \geq 0.5$, i.e. if $z \geq 0$).
- It's smooth and differentiable everywhere, which is required for the gradient-based optimization we'll derive below.

The Sigmoid's Derivative

$$\sigma'(z) = \frac{d}{dz} \left[\frac{1}{1 + e^{-z}} \right] = \frac{e^{-z}}{(1 + e^{-z})^2}$$

Rewrite $\frac{e^{-z}}{(1+e^{-z})^2}$ as $\frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}}$. The first factor is just $\sigma(z)$. For the second factor, add and subtract 1 in the numerator: $\frac{e^{-z}}{1+e^{-z}} = \frac{(1+e^{-z})-1}{1+e^{-z}} = 1 - \frac{1}{1+e^{-z}} = 1 - \sigma(z)$. Multiplying the two pieces back together:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Key Insight

The sigmoid's derivative has a beautifully simple, self-referential form: it can be written purely in terms of $\sigma(z)$ itself, with no need to recompute an exponential. This convenient fact will do most of the heavy lifting in the gradient derivation of Section 3.7.

3.3 Model Representation & Log-Odds

The logistic regression model predicts the probability that $y = 1$, given x :

$$\hat{y} = p(y = 1 \mid x; \theta) = \sigma(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

Equivalently, the **log-odds** (logit) of the outcome is *linear* in the parameters:

$$\log \left(\frac{p}{1-p} \right) = \theta^T x, \quad \text{where odds} = \frac{p}{1-p}$$

Worked Example

If $p = 0.8$, the odds are $\frac{0.8}{0.2} = 4$ — “four times as likely to be 1 as to be 0.” If a fitted coefficient is $\theta_j = 0.7$ for some feature x_j , a one-unit increase in x_j increases the log-odds by 0.7, which means it multiplies the odds by $e^{0.7} \approx 2.01$ — roughly doubling the odds of the positive class for every one-unit increase in that feature.

Key Insight

Logistic regression is a **Generalized Linear Model (GLM)**: it keeps the familiar $\theta^T x$ linear core from Chapter 2, but wraps it with the logit link function. Each θ_j has a direct, interpretable meaning: a one-unit increase in x_j changes the log-odds by θ_j , i.e. it multiplies the odds by e^{θ_j} .

3.4 Maximum Likelihood: The Bernoulli Model

Just as Chapter 2's squared-error loss fell out of a Gaussian noise assumption, logistic regression's loss falls out of a **Bernoulli** assumption on the labels. Each label $y \in \{0, 1\}$ is modeled as a coin flip with success probability $\hat{y}$:

$$p(y \mid x; \theta) = \hat{y}^y (1 - \hat{y})^{1-y}$$

Plain-English Intuition

Substitute $y = 1$ into this formula and it collapses to $\hat{y}^1 (1 - \hat{y})^0 = \hat{y}$. Substitute $y = 0$ and it collapses to $\hat{y}^0 (1 - \hat{y})^1 = 1 - \hat{y}$. A single algebraic expression elegantly covers both cases at once, which is exactly what makes the upcoming derivation so clean.

Assuming the m training examples are independent, the full-dataset likelihood is their product:

$$L(\theta) = \prod_{i=1}^m \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i}$$

Our goal is to find θ that maximizes $L(\theta)$ — the parameters that make the observed labels most probable under our model. As in Chapter 2, we take the log first to turn the product into a sum.

3.5 From Log-Likelihood to Binary Cross-Entropy Loss

Step 1 — Take the log of the likelihood

$$\ell(\theta) = \log L(\theta) = \sum_{i=1}^m \log [\hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i}] = \sum_{i=1}^m [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

(using $\log(a^b) = b \log a$ and $\log(ab) = \log a + \log b$ to split each term apart).

Step 2 — Maximizing log-likelihood $\Leftrightarrow$ minimizing cross-entropy

Maximizing $\ell(\theta)$ is equivalent to minimizing $-\ell(\theta)$, and dividing by the positive constant m (to average, exactly as in Chapter 2) doesn't change the optimal θ . This defines the **cross-entropy** (a.k.a. "log loss") cost:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

Key Insight

Minimizing cross-entropy is mathematically identical to maximum likelihood estimation. Unlike squared error composed with a sigmoid (which is non-convex and can trap gradient descent in a bad local minimum), cross-entropy *is* convex in θ (proved in Section 3.8), so gradient descent is guaranteed to find the global minimum — exactly the same guarantee Chapter 2 established for MSE, now carried over to classification.

3.6 Why Cross-Entropy Makes Intuitive Sense

For a single example, the cost splits into two cases depending on the true label y :

$$\text{cost}(\hat{y}, y=1) = -\log(\hat{y}) \qquad \text{cost}(\hat{y}, y=0) = -\log(1 - \hat{y})$$

- When $y = 1$: cost $\rightarrow 0$ as $\hat{y} \rightarrow 1$ (confident and correct), but cost $\rightarrow \infty$ as $\hat{y} \rightarrow 0$ (confident and *wrong*).
- When $y = 0$: the mirror image — being confidently wrong is punished severely.

Worked Example

If the true label is $y = 1$ and the model predicts $\hat{y} = 0.9$ (confident and correct), the cost is $-\log(0.9) \approx 0.105$ — small. If instead the model predicts $\hat{y} = 0.1$ (confident and *wrong*), the cost is $-\log(0.1) \approx 2.303$ — over 20 times larger, even though both predictions are “only” 0.8 away from the opposite extreme in a naive sense. This asymmetric blow-up as $\hat{y} \rightarrow 0$ is exactly what makes cross-entropy so effective at discouraging confidently wrong predictions.

Key Insight

This asymmetric, unbounded penalty is exactly what drives the model to push predictions toward the correct extreme, and to be genuinely cautious about ever being confidently wrong.

3.7 The Gradient — A Beautifully Simple Result

We need $\partial J / \partial \theta_j$. Since J depends on θ_j only through $\hat{y}_i = \sigma(\theta^T x_i)$, the chain rule applies:

$$\frac{\partial J}{\partial \theta_j} = \sum_{i=1}^m \frac{\partial J}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial \theta_j}$$

Working out each piece of the chain rule

Piece 1: differentiating $J = -\frac{1}{m} \sum_i [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$ with respect to $\hat{y}_i$ (treating each example separately, dropping the outer sum and $\frac{1}{m}$ for a moment):

$$\frac{\partial}{\partial \hat{y}_i} [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)] = \frac{y_i}{\hat{y}_i} - \frac{1 - y_i}{1 - \hat{y}_i}$$

Piece 2: since $\hat{y}_i = \sigma(z_i)$ where $z_i = \theta^T x_i$, the chain rule and the sigmoid derivative from Section 3.2 give:

$$\frac{\partial \hat{y}_i}{\partial \theta_j} = \sigma'(z_i) \cdot \frac{\partial z_i}{\partial \theta_j} = \hat{y}_i(1 - \hat{y}_i) \cdot x_{ij}$$

Multiplying the two pieces together — and watch how much cancels:

$$\left(\frac{y_i}{\hat{y}_i} - \frac{1 - y_i}{1 - \hat{y}_i} \right) \hat{y}_i(1 - \hat{y}_i)x_{ij} = [y_i(1 - \hat{y}_i) - (1 - y_i)\hat{y}_i]x_{ij}$$

Expanding the bracket: $y_i - y_i\hat{y}_i - \hat{y}_i + y_i\hat{y}_i = y_i - \hat{y}_i$. The $y_i\hat{y}_i$ terms cancel exactly, leaving:

$$\left(\frac{y_i}{\hat{y}_i} - \frac{1 - y_i}{1 - \hat{y}_i}\right) \hat{y}_i(1 - \hat{y}_i)x_{ij} = (y_i - \hat{y}_i)x_{ij}$$

Restoring the outer sum, the $\frac{1}{m}$, and remembering we differentiated J (which has an overall minus sign in front), the two minus signs cancel:

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)x_{ij} \quad \implies \quad \nabla_{\theta} J(\theta) = \frac{1}{m} X^T (\hat{y} - y)$$

Key Insight

This is *exactly* the same formula as the linear regression gradient from Section 2.5 — $\nabla_{\theta} J(\theta) = \frac{1}{m} X^T (\hat{y} - y)$ — just with $\hat{y}_i$ now coming from the sigmoid instead of a raw linear output. This elegant symmetry is a general property of GLMs equipped with a “canonical link” function, and it’s exactly why logistic regression’s gradient descent code looks almost identical to linear regression’s.

3.8 Optimization & the Convexity Guarantee

The gradient descent update rule mirrors Chapter 2 exactly:

$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j}$$

Unlike squared error composed with a sigmoid (which is non-convex), cross-entropy’s **Hessian** (the matrix of second derivatives) can be shown to be positive semi-definite everywhere — the formal definition of convexity for a smooth, twice-differentiable function. Because of this, the exact same descent-lemma argument from Section 2.8 applies here too: gradient descent’s steps shrink monotonically, and θ_t converges to the unique global minimum.

In practice, ‘scikit-learn’¹’s `LogisticRegression` typically uses smarter optimizers than plain gradient descent:

- **Newton’s Method / IRLS** (Iteratively Reweighted Least Squares) uses the Hessian for a curvature-aware step, converging in far fewer iterations at a higher cost per step.
- **L-BFGS / Conjugate Gradient** — quasi-Newton methods that approximate the Hessian cheaply, and are ‘scikit-learn’¹’s default solvers for fast, robust convergence.

3.9 Regularized & Multiclass (Softmax) Logistic Regression

Exactly as in Chapter 2, logistic regression can be **L2-regularized**:

$$J_{\text{ridge}}(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right] + \lambda \sum_j \theta_j^2$$

3.9.1 Extending to More Than Two Classes: Softmax Regression

For K classes, each class k gets its own parameter vector θ_k , and the predicted probability of class k is given by the **softmax function**:

$$p(y = k | x) = \frac{e^{\theta_k^T x}}{\sum_{j=1}^K e^{\theta_j^T x}}$$

Plain-English Intuition

Softmax takes K raw linear scores (one per class) and converts them into a valid probability distribution: every output is positive (thanks to the exponential), and dividing by the sum of all K exponentials forces everything to add up to exactly 1. The class with the highest raw score ends up with the highest probability, but every class still gets a nonzero share.

The loss generalizes to **categorical cross-entropy**:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K \mathbb{1}\{y_i = k\} \log p(y_i = k | x_i)$$

where $\mathbb{1}\{\cdot\}$ is 1 if the condition is true and 0 otherwise — so for each example, only the log probability assigned to its *true* class contributes to the sum.

Key Insight

Setting $K = 2$ and simplifying the softmax formula recovers the binary sigmoid exactly. **Logistic regression is the special case of softmax regression with two classes** — one more instance of the same underlying idea reappearing in a more general form.

3.10 Evaluating a Classification Model

$$\text{Accuracy} = \frac{TP + TN}{\text{total}} \quad \text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN} \quad F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

where TP = true positives, TN = true negatives, FP = false positives, FN = false negatives.

Plain-English Intuition

Picture a spam filter. A **true positive** is spam correctly caught. A **false positive** is a legitimate email wrongly flagged as spam (annoying — you might miss an important message). A **false negative** is spam that slipped through to your inbox (annoying in a different way). **Precision** asks “of everything I flagged as spam, how much really was spam?” **Recall** asks “of all the actual spam, how much did I catch?” These two goals are often in tension, which is exactly why we track both.

- **Accuracy** is the overall fraction correct. It is misleading on imbalanced data — e.g. a dataset that’s 95% “healthy” patients can score 95% accuracy just by always predicting “healthy,” without learning anything useful.
- **Precision** matters most when false positives are costly (e.g. wrongly flagging a legitimate transaction as fraud).

- **Recall** matters most when false negatives are costly (e.g. missing an actual disease diagnosis).
- **F1** is the harmonic mean of precision and recall — a single number useful when you care about both roughly equally, and it penalizes a model that sacrifices one metric entirely to inflate the other.

Worked Example

Suppose a disease-screening classifier is tested on 100 patients, 10 of whom actually have the disease. It predicts 12 patients as positive, correctly identifying 8 of the 10 true cases (so $TP = 8$, $FN = 2$, $FP = 4$, $TN = 86$).

$$\text{Accuracy} = \frac{8 + 86}{100} = 94\%, \quad \text{Precision} = \frac{8}{8 + 4} = 66.7\%, \quad \text{Recall} = \frac{8}{8 + 2} = 80\%$$

Despite a reassuring-looking 94% accuracy, precision reveals that a third of the “positive” flags were false alarms, and recall reveals that one in five real cases was still missed — both details the single accuracy number hides completely.

ROC-AUC (Receiver Operating Characteristic — Area Under the Curve) is a threshold-independent measure of separability. Instead of fixing the decision threshold at 0.5, the ROC curve plots the true positive rate against the false positive rate as the threshold sweeps across every possible value from 0 to 1. The **area under this curve** summarizes performance across all thresholds at once: an AUC of 1.0 is a perfect classifier, 0.5 is equivalent to random guessing, and values in between interpolate smoothly — higher is better.

Watch Out

Always match your evaluation metric to the actual cost of different mistakes in your problem. A model tuned purely for accuracy on an imbalanced dataset can look excellent on paper while being nearly useless in practice.

Chapter 4

Synthesis: Linear and Logistic Regression, Side by Side

4.1 Comparison Table

	Linear Regression	Logistic Regression
Task	Regression (continuous y)	Classification (categorical y)
Output	$h_{\theta}(x) = \theta^T x \in \mathbb{R}$	$\sigma(\theta^T x) = p(y=1 \mid x) \in (0, 1)$
Link function	Identity	Sigmoid / Logit
Loss function	Mean Squared Error	Binary Cross-Entropy
Derived from	Gaussian likelihood (MLE)	Bernoulli likelihood (MLE)
Closed form?	Yes — Normal Equation	No — needs iterative optimization
Gradient formula	$\frac{1}{m} X^T (X\theta - y)$	$\frac{1}{m} X^T (\hat{y} - y)$ — same shape!
Regularization	Ridge / Lasso / Elastic Net	L2 / L1 penalty on log-loss
Key metrics	R^2 , RMSE, MAE	Accuracy, F1, ROC-AUC

4.2 Key Takeaways

1. Both models are **Generalized Linear Models**: a linear core $\theta^T x$, wrapped by a link function (identity for linear regression, sigmoid for logistic regression).
2. Their loss functions are **not arbitrary** — MSE and cross-entropy both fall directly out of Maximum Likelihood Estimation, under Gaussian and Bernoulli noise assumptions respectively (Chapters 2 and 3).
3. Both gradients reduce to the **same elegant form**: $\frac{1}{m} X^T (\text{prediction} - \text{label})$ — which is exactly why the gradient descent code for the two models is nearly identical.
4. Linear regression has a **closed-form solution** (the Normal Equation); logistic regression needs iterative optimization because there is no closed form for its likelihood equations.
5. **Regularization** (Ridge / Lasso / Elastic Net, or the C parameter in scikit-learn) extends naturally to both models to control overfitting.
6. Always **match your evaluation metric to the problem**: R^2 /RMSE/MAE for regression; Precision/Recall/F1/ROC-AUC for classification, especially under class imbalance.

4.3 From Theory to a Working Project

The companion Jupyter notebook puts every derivation in this guide into runnable code, on two hands-on projects that mirror the structure of this guide exactly:

- **House Price Predictor** (Linear Regression) — engineer and scale numeric features, fit via both the Normal Equation and Gradient Descent and compare them, evaluate with R^2 /RMSE/MAE, and apply Ridge/Lasso when features are correlated.
- **Iris Species Classifier** (Logistic Regression) — binary and multiclass (softmax) classification, visualizing the sigmoid-derived decision boundary, evaluating with a confusion matrix, precision/recall/F1 and ROC-AUC, and regularizing to avoid overfitting on a small dataset.

Every code cell in the notebook is preceded by a markdown explanation connecting it back to the exact derivation in this guide, and every plot is followed by a short discussion of what it shows — so the two documents are meant to be read side by side.

Appendix A: Notation & Symbol Glossary

Symbol	Meaning
m	Number of training examples
n	Number of input features
$x^{(i)}$	Feature vector of the i -th training example
$y^{(i)}$	True target value of the i -th training example
θ	Parameter (weight) vector, including bias θ_0
$h_{\theta}(x)$	The model's hypothesis function (prediction), $= \theta^T x$ for linear regression
$\hat{y}$	Predicted value or predicted probability
X	Design matrix, $m \times (n+1)$, one row per example
$J(\theta)$	Cost function
α	Learning rate
$\nabla_{\theta} J(\theta)$	Gradient of the cost function with respect to θ
$\sigma(z)$	Sigmoid function, $= \frac{1}{1+e^{-z}}$
λ	Regularization strength
L	Lipschitz constant of the gradient (used in convergence theory)
$\mathcal{N}(\mu, \sigma^2)$	Normal (Gaussian) distribution with mean μ , variance σ^2
$L(\theta)$	Likelihood function
$\ell(\theta)$	Log-likelihood function
K	Number of classes (multiclass / softmax regression)
TP, TN, FP, FN	True/false positives/negatives

Appendix B: Frequently Asked Questions

Q: Why divide by $2m$ instead of just m in the cost function?

The $\frac{1}{m}$ is for averaging; the extra factor of 2 is pure convenience so that differentiating the square term cancels it exactly, leaving a clean gradient formula. Neither choice changes which θ minimizes the cost.

Q: Do I always need to scale my features?

For gradient descent, yes — strongly recommended, since it makes the cost surface more circular and lets you safely use a larger learning rate (Section 2.9 and the Convergence Theory discussion in Section 2.8 explain why). For the Normal Equation, scaling is not mathematically necessary, since it solves for the exact optimum regardless of scale.

Q: Is logistic regression secretly doing regression?

Only in the sense that it computes a linear combination $\theta^T x$ internally (the “regression” part), which is then passed through the sigmoid to produce a probability. The final output and the loss function are both built for classification, not for predicting a continuous number.

Q: Why can’t I just use accuracy for every classification problem?

Accuracy treats every mistake identically and ignores class imbalance. A model that always predicts the majority class can achieve high accuracy while being completely useless in practice. Always check precision, recall, and the confusion matrix, especially when one class is rarer than the other.

Q: What happens if $X^T X$ is not invertible?

The Normal Equation from Section 2.6 fails outright. This typically happens with perfectly correlated (redundant) features, or when you have more features than training examples. Ridge regression (Section 2.10) fixes this directly, since adding λI to $X^T X$ always makes the result invertible.

Q: Which regularizer should I use — Ridge, Lasso, or Elastic Net?

Start with Ridge if you believe most of your features are genuinely useful. Reach for Lasso if you suspect many features are irrelevant and want automatic feature selection. Use Elastic Net when your features are correlated with each other and you want Lasso’s sparsity without its instability under correlation.

Appendix C: Further Study

The ideas in this guide extend directly into topics covered in later weeks:

- **Polynomial regression** — reusing the exact same linear regression machinery on polynomial features (x^2 , x^3 , interaction terms) to fit curved relationships.
- **Generalized Linear Models (GLMs)** more broadly — Poisson regression for count data, and other link functions, all built on the same MLE-driven recipe used in Chapters 2 and 3.
- **Neural networks** — a stack of logistic-regression-like units, where the final layer's softmax and cross-entropy loss are exactly the multiclass formulas derived in Section 3.9.
- **Support Vector Machines** — an alternative approach to classification with a different loss function (hinge loss) but a similar linear-decision-boundary intuition to logistic regression.
- **Regularization paths and cross-validation** — systematic methods for choosing λ (or C) rather than guessing, which is the standard practice in real projects.

Key Insight

The single most valuable habit to take away from this guide is not any individual formula, but the **recipe itself**: state a probabilistic assumption about your data $\rightarrow$ derive a loss function via Maximum Likelihood Estimation $\rightarrow$ derive its gradient $\rightarrow$ optimize. This recipe reappears, almost unchanged, throughout the rest of machine learning.